

Recursion, Fractals and space-filling curves in Logo

Alina Musaeva

Julian Schwarzer

Lars Wittner

27.07.2026

Abstract

Yeah its a paper about logo i guess lol xD [1].

Contents

1	Introduction	2
2	Recursion	2
2.1	Concept	2
2.2	Example: Pythagoras tree	3
3	Fractals	5
3.1	Concept	5
3.2	Example: Koch flake	5

1 Introduction

...

2 Recursion

In addition to these basic commands we are now capable of writing some actual procedures. For that we first have to introduce the concept of recursion and fractals which make up a large part of Logo as a functional programming language.

2.1 Concept

Recursion has its origin in the latin word *recurrere* (*to go back*). A recursive written function has to contain three key parts:

1. Base case
2. Calling itself
3. Recursive step

The base case will be a condition that indicates on what case our recursive calls will end up terminating. We need this due to our limitations of storage and to avoid an infinite amount of calls on our call stack. To actually reach this base case we need a recursive step that needs to be provided by our recursive call that approaches our base case.

Fibonacci numbers

```
1 private static int fib(int n) {  
2     if (n <= 0) return -1; // wrong input  
3     if (n == 1) return 0; // base case 1  
4     if (n == 2) return 1; // base case 2  
5     return fib(n - 1) + fib(n - 2); // recursive call  
6 }
```

java

In this java implementation of the Fibonacci numbers you can see that there are also multiple base cases possible because the first Fibonacci number has to be 0 and the second one needs to be 1. After these base cases the recursive call with the recursive steps are embedded in the returned value of this method. As you might notice there might be a problem by this implementation.

Very important is that the recursive step actually reaches our base case. In this example each reduction ($n - 1$ and $n - 2$) will either reach our first or our second base case and terminate. If we would have skipped our base case and not had filtered out our wrong inputs ($n \leq 0$) we would not terminate and the recursion will end up in an infinite loop.

The phrase *depth of recursion* determines the amount of nested recursive functions inside of itself.

Recursion often finds usage in science like mathematics or computer science *eg. factorials, Fibonacci*.

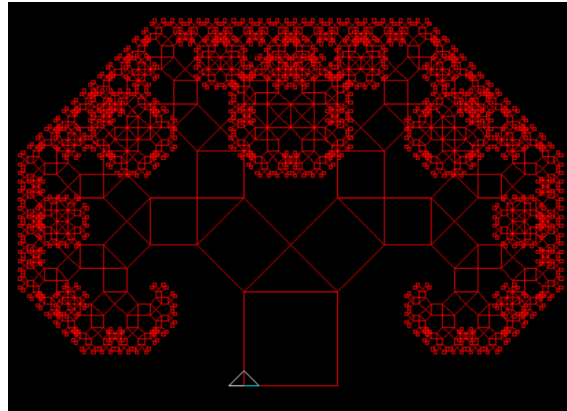


Figure 1: Pythagoras tree

2.2 Example: Pythagoras tree

The pythagoras tree is a very common example for explaining recursion. Its main shape is a square with recursively placed squares on top of the base square rotated by a predefined angular that calculates based on the famous Pythagoras formula (1).

To implement this we will use the previously explained scheme: We declare our procedure within the first line (keyword *to*) with its name being *pythagoras* an two needed parameters (indicated by the colons) as *length* and *depth* and end our definition with the keyword *end* in our last line.

```
1  to pythagoras :length :depth
2      ...
3  end
```

logo

After this we have our main form drawn (the square) which will be drawn by repeating four times going forward and then turning right.

```
1  repeat 4 [fd :length rt 90]
```

logo

Now we write down our base case what translates to our program to end the procedure if we reach a depth of 0. The depth represents the maximum amount of squares placed on top of another square (recursion depth).

```
1  if :depth = 0 [stop]
```

logo

We end up at our starting position at the left bottom corner of the just drawn square. To draw our recursive squares we need to correct our position. For that we are going forward by our length and continue by rotating to the left by 45 degree which will be our predefined rotation so both squares will have the exact same length and our Pythagoras tree becomes axially symmetric.

```
1  fd :length lt 45
```

logo

Our recursive squares are then drawn with a reduced length in the scale of the Pythagoras formula and a depth that decreases by 1 which ensures the base case of depth 0 will be reached at some point.

```
1  pythagoras :length / (sqrt 2) :depth - 1
```

logo

Because we use an angle of 45 degree the following derivation explains the new length:
We assume our hypotenuse as a and our two cathets as length b .

According to Pythagoras:

$$\begin{aligned} a^2 + a^2 &= b^2 \\ 2a^2 &= b^2 \\ \sqrt{2}a &= b \\ b &= \frac{a}{\sqrt{2}} \end{aligned}$$

Now we need to correct our position again to resume with the second square.

```
1  rt 90 fd :length / (sqrt 2)
```

logo

Now at the right starting point for our last square, we perform the same actions as our first square.

```
1  pythagoras :length / (sqrt 2) :depth - 1
```

logo

After each recursive depth we return to the position of the previous one ($depth + 1$).

```
1  bk :length / (sqrt 2) lt 45 bk :length
```

logo

Our resulting procedure will result in the following:

Pythagoras tree

```
1  to pythagoras :length :depth ; variable length and depth
2  repeat 4 [fd :length rt 90] ; square
3  if :depth = 0 [stop] ; base case
4  fd :length lt 45 ; change pos
5  pythagoras :length / (sqrt 2) :depth - 1 ; recursive left
6  rt 90 fd :length / (sqrt 2) ; change pos
7  pythagoras :length / (sqrt 2) :depth - 1 ; recursive right
8  bk :length / (sqrt 2) lt 45 bk :length ; go back
9  end
```

logo

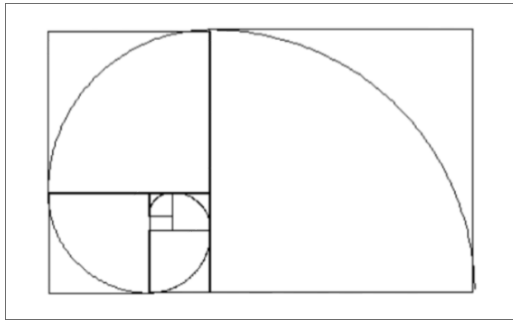


Figure 2: Golden ratio

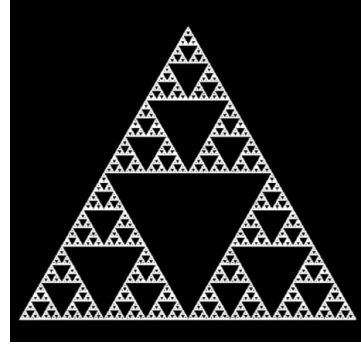


Figure 3: Sierpiński triangle

3 Fractals

3.1 Concept

Fractals are the idea of performing a recursion with a *recursion depth* of infinity. Like recursion, fractals also have their origin in a latin word (*fracere (to break)*).

Fractals are a theoretical concept that cannot be used in reality again due to eg. the finite resolution a monitor has.

The main property of fractals are the *self-similarity* which means you can identify the fractal itself as a part of itself by imagining to zoom infinitely in.

In order to be able to work with fractals nonetheless we can use visualisation by recursion with a relatively high recursion depth a . The depth of recursion is the scale of sharpness a fractal image has.

3.2 Example: Koch flake

The Koch flake became quite popular for its simplistic form and minimal lines of code for a quite nice looking shape of a snowflake. First described by Helge von Koch in 1904 is made up of three triangular shaped Koch curves. These Koch curves follow simple rules. At first a Koch curve simply is a line with a length. By increasing the depth there will be placed Koch flakes on top of this line forming a triangle on top of the line. The higher the depth the more triangles are placed on top of each other.

We will declare our Koch curve in the same way as we did with the Pythagoras tree.

```
1  to curve :length :depth
2    ...
3  end
```

logo

We will start by defining our termination condition (our *base case*) as drawing a *length*-long straight line.

Part 1/4:

```
1  ; base case: if depth is 0, move forward and stop
2  if :depth = 0 [fd :length stop]
```

logo

For achieving our Koch curve with a triangular shape placed on a line, our Koch curve consists of four parts: Now we write down our base case what translates to our program to end the procedure if we reach a depth of 0. The depth represents the maximum amount of squares placed on top of another square (recursion depth).

```

1  ; fractal part 1/4 (move)
2  curve :length / 3 :depth - 1
3  ; fractal part 2/4 (turn left and move)
4  lt 60
5  curve :length / 3 :depth - 1
6  ; fractal part 3/4 (turn right and move)
7  rt 120
8  curve :length / 3 :depth - 1
9  ; fractal part 4/4 (turn left and move)
10 lt 60
11 curve :length / 3 :depth - 1

```

logo

So now we have our Koch curve to get the shape of a snowflake we simply need to expend it by another procedure *flake* that repeats our *curve* three times and between each curve it rotates the turtle so we have a shape of a triangle.

Kock flake

```

1  ; procedure with parameters length and depth
2  to curve :length :depth
3  ; base case: if depth is 0, move forward and stop
4  if :depth = 0 [fd :length stop]
5  ; fractal part 1/4 (move)
6  curve :length / 3 :depth - 1
7  ; fractal part 2/4 (turn left and move)
8  lt 60
9  curve :length / 3 :depth - 1
10 ; fractal part 3/4 (turn right and move)
11 rt 120
12 curve :length / 3 :depth - 1
13 ; fractal part 4/4 (turn left and move)
14 lt 60
15 curve :length / 3 :depth - 1
16 end
17
18 to flake :depth
19 ; 3 sides of base triangle form
20 repeat 3 [
21   curve 200 :depth
22   rt 120 ; turn right to next triangle base side
23 ]
24 end

```

logo

References

- [1] Juraj Hromkovic. *Einführung in die Programmierung mit LOGO*. Vol. 206. Springer, 2010.